

Mimari Tasarım Raporu

KARADENİZ TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



PROJENİN KONUSU:

2D metroidvania türü bilgisayar tabanlı macera odaklı, çocuk ve yetişkin herkesin oynayabileceği oyun geliştirmek.

Yazılım Mühendisliği Mimari Tasarım Raporu

Alperen ALTUNEL- 425468

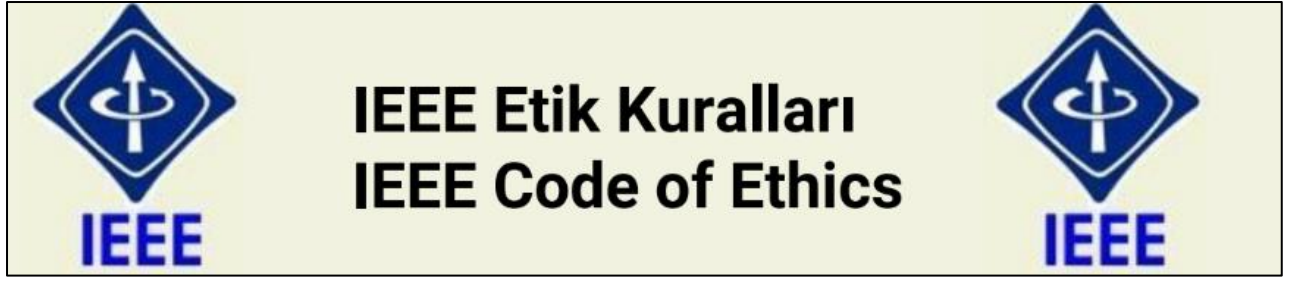
Berat KAYAR- 425500

Eren KUŞDİL-425429

Özer KOCA- 425472

Alaattin UYSAL-425492

2024-2025 BAHAR DÖNEMİ



Mesleğime karşı şahsi sorumluluğumu kabul ederek, hizmet ettiğim toplumlara ve üyelerine en yüksek etik ve mesleki davranışta bulunmaya söz verdiğimi ve aşağıdaki etik kurallarını kabul ettiğimi ifade ederim:

- Kamu güvenliği, sağlığı ve refahı ile uyumlu kararlar vermenin sorumluluğunu kabul etmek ve kamu veya çevreyi tehdit edebilecek faktörleri derhal açıklamak;
- Mümkün olabilecek çıkar çatışması, ister gerçekten var olması isterse sadece algı olması, durumlarından kaçınmak. Çıkar çatışması olması durumunda, etkilenen taraflara durumu bildirmek;
- Mevcut verilere dayalı tahminlerde ve fikir beyan etmelerde gerçekçi ve dürüst olmak;
- Her türlü rüşveti reddetmek;
- Mütenasip uygulamalarını ve muhtemel sonuçlarını gözeterek teknoloji anlayışını geliştirmek;
- Teknik yeterliliklerimizi sürdürmek ve geliştirmek, yeterli eğitim veya tecrübe olması veya işin zorluk sınırları ifade edilmesi durumunda ancak başkaları için teknolojik sorumlulukları üstlenmek;
- Teknik bir çalışma hakkında yansız bir eleştiri için uğraşmak, eleştiriye kabul etmek ve eleştiriye yapmak; hatları kabul etmek ve düzeltmek; diğer katkı sunanların emeklerini ifade etmek;
- Bütün kişilere adilane davranmak; ırk, din, cinsiyet, yaş, milliyet, cinsi tercih, cinsiyet kimliği, veya cinsiyet ifadesi üzerinden ayrımcılık yapma durumuna girişmemek;
- Yanlış veya kötü amaçlı eylemler sonucu kimsenin yaralanması, mülklerinin zarar görmesi, itibarlarının veya istihdamlarının zedelenmesi durumlarının oluşmasından kaçınmak;
- Meslektaşlara ve yardımcı personele mesleki gelişimlerinde yardımcı olmak ve onları desteklemek.

IEEE Yönetim Kurulu tarafından Ağustos 1990'da
onaylanmıştır

İÇİNDEKİLER

	Sayfa No
IEEE ETİK KURALLARI	2
İÇİNDEKİLER	3
1. Giriş	4
2. Sistem Genel Bakış	4
2.1 Sistemler	4
2.1.1 Menu Sistemi	4
2.1.2 Oynanış Sistemi	5
2.1.3 Durum Yönetim Sistemi	5
2.1.4 Varlık Yönetim Sistemi	5
2.1.5 Oyun Yönetim Sistemi	6
2.1.6 Ayarlar Sistemi	6
2.2 Alt Sistemler	6
3. Sınıf Tanımları	8
4. Rutinler ve Arayüzler	12
5. Dil, Çerçeve ve Kodlama Standartları	13
6. Test Yaklaşımları	14
7. Mimari Diyagram	16
8. Sonuç	16

1. Giriş

Bu rapor, Pygame kullanılarak geliştirilen 2D bir oyun için mimari tasarımı özetlemektedir. Oyunumuzu adı The Way'dir. Proje şu anda 4 ana modülden oluşmaktadır: menu.py (oyun menü sistemini yönetir), player.py (temel oyun mekaniklerini uygular), enemy.py (Düşman mekaniklerini ayarlar), platform.py (Harita ve seviye mekaniklerini ayarlar). Amaç, bu bileşenleri sorunsuz bir şekilde entegre eden, performansı artıran ve gelecekteki genişlemeleri destekleyen ölçeklenebilir ve sürdürülebilir bir mimari oluşturmaktır. Tasarım, aşağıdaki kriterlere uygundur:

- Modüler sistem ve alt sistem tasarımı.
- Açık sınıf tanımları, rutinler ve arayüzler.
- Dil, çerçeve ve kodlama standartlarının belirtilmesi.

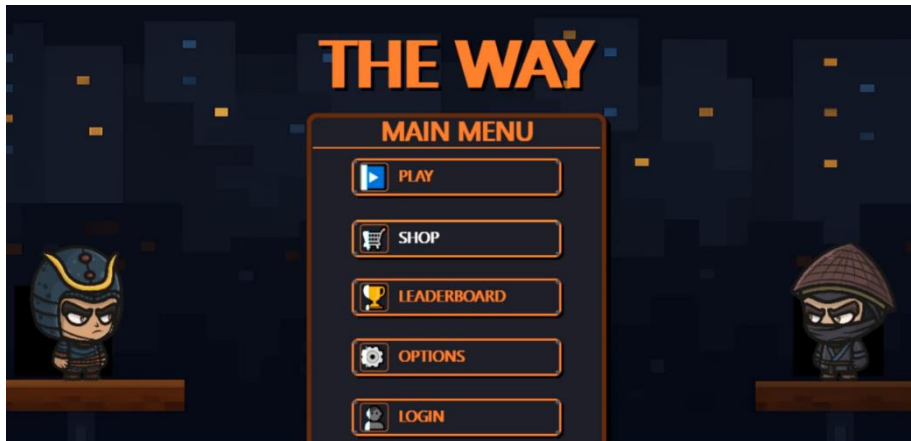
2. Sistem Genel Bakış

The Way, oyuncuların bir oyun karakterini kontrol ettiği, haritalar arası geçiş yaptığı, düşmanlarla savaştığı ve ayarlar, alışveriş ve seviye seçimi için bir menü sistemiyle etkileşim kurduğu bir 2D platform oyunudur. Mimari, oyun deneyimini sunmak için etkileşimde bulunan yüksek seviye sistemlere bölünmüştür.

2.1 Sistemler

2.1.1 Menü Sistemi:

- Ana menü, ayarlar, alışveriş, seviye seçimi ve giriş ekranları dahil olmak üzere oyunun kullanıcı arayüzünü yönetir.
- Gezinti, ayar ayarlamaları (örneğin ses seviyesi, parlaklık) ve alışveriş işlemleri için kullanıcı girişlerini işler.
- Oyun oynanışına geçişi, oynanış sistemini başlatmak için sağlar.



2.1.2 Oynanış Sistemi:

- Oyuncunun hareketi, savaş, düşman yapay zekâsı ve harita işleme gibi temel oyun mekaniklerini uygular.
- Oyun içi etkileşimleri (örneğin iksir toplama, düşman karşılaşmaları, seviye geçişleri) yönetir.

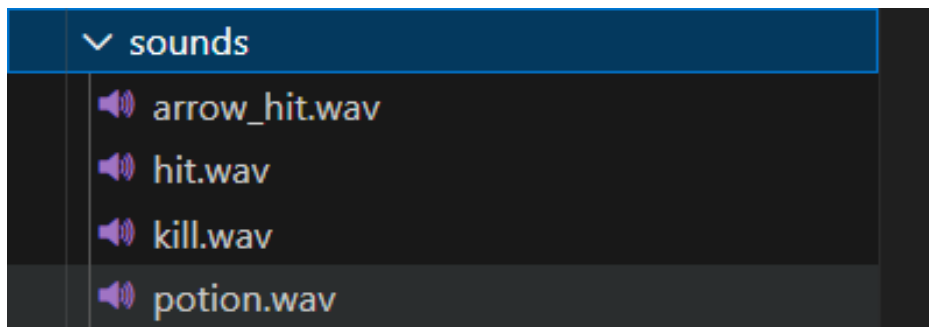


2.1.3 Durum Yönetimi Sistemi:

- Oyunun durumunu (örneğin oyuncu sağlığı, jetonlar, satın alınan yetenekler, ayarlar) merkezi bir şekilde yönetir ve sistemler arasında tutarlılık sağlar.
- Geçişler sırasında durumun kalıcılığı için seri hale getirme ve ters seri hale getirme sağlar.

2.1.4 Varlık Yönetimi Sistemi:

- Oyun varlıklarının (görüntüler, sesler, haritalar) yüklenmesini ve önbelleğe alınmasını yöneterek performansı optimize eder ve hata dayanıklılığını sağlar.

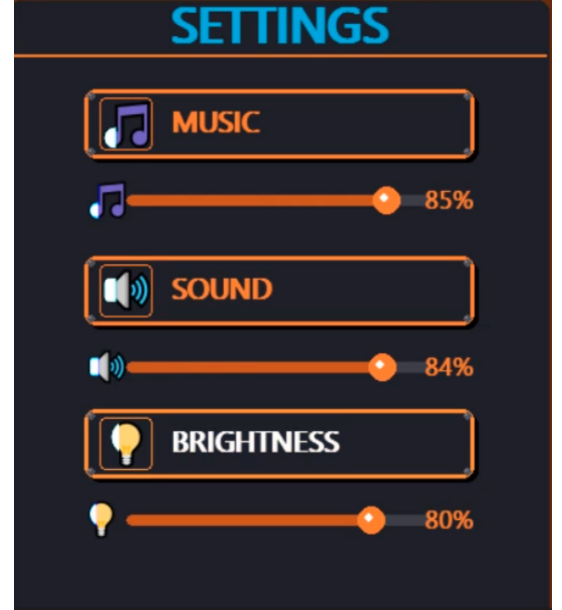
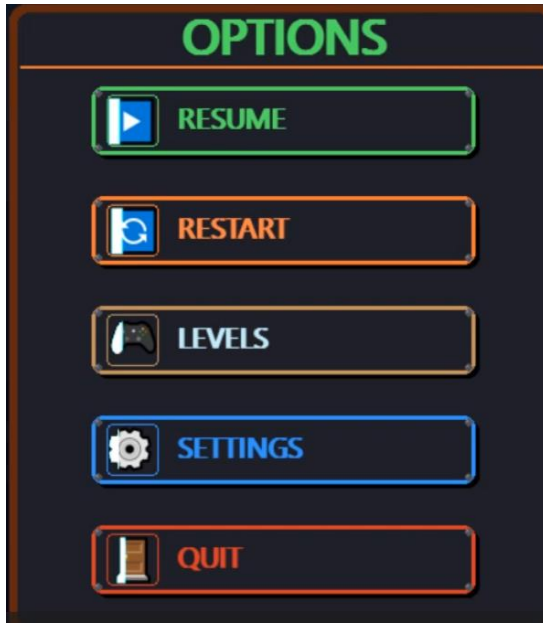


2.1.5 Oyun Yönetimi Sistemi:

- Menü ve oynanış sistemleri arasındaki geçişleri yönetir, uygun yaşam döngüsü yönetimi ve durum aktarımı sağlar.

2.1.6 Ayarlar Sistemi:

- Ses seviyesi, parlaklık, yakınlaştırma gibi oyun ayarlarını merkezi bir şekilde yönetir ve bunları menü ile oynanış sistemleri arasında uygular.



2.2 Alt Sistemler

Her sistem, modülerlik için alt sistemlere ayrılmıştır:

• Menü Sistemi:

- **UI İşleme Alt Sistemi:** Düğmeler, kaydırmalı çubuklar, metin giriş kutuları ve arka plan görüntülerini işler.
- **Giriş İşleme Alt Sistemi:** Gezinti ve etkileşimler için kullanıcı girişlerini işler.
- **Durum Geçiş Alt Sistemi:** Oyun oynanışına geçişi, yönetim sistemiyle koordine eder.

Oynanış Sistemi:

- **Oyuncu Alt Sistemi:** Samurai sınıfını hareket, savaş ve sağlık için yönetir.

```
# Samurai (Player) class
class Samurai(pygame.sprite.Sprite):
    def __init__(self, walk_spritesheet, idle_spritesheet, jump_spritesheet, ...):
        # ...

    def update_grab_area(self):
        """Tutma alanını güncelle (karakterin üst kısmında bir alan)"""
        self.grab_area.midbottom = (self.hitbox.centerx, self.hitbox.top - 5)

    def check_wall_grab(self, collision_rects):
        """Duvarı tutup tırmanma kontrolü"""
        if self.is_climbing or self.is_hurt or self.is_dead or self.on_ground:
            return None
```

- **Düşman Alt Sistemi:** NinjaMonk için devriye, saldırı ve ölüm davranışlarını uygular.

```
# NinjaMonk sınıfı - Devriye gezen ve saldıran düşman
class NinjaMonk(pygame.sprite.Sprite):
    def __init__(self, idle_spritesheet, walk_spritesheet, attack_spritesheet, ...):
        # ...

    # Mevcut metodlar korunuyor, sadece devriye ve zıplama için yenileri ekleniyor
    def platform_kontrolu(self, collision_rects):
        """Platformun kenarında zıplama kontrolü"""
        simdiki_zaman = pygame.time.get_ticks()
        if simdiki_zaman - self.son_ziplama_zamani < self.ziplama_bekleme_suresi:
            return

        # Düşman hangi yöne bakıyorsa o yönde kenar kontrolü yap
        yon = -1 if self.flip else 1
```

- **Harita Alt Sistemi:** pytmx kullanarak harita yükleme, işleme, çarpışmalar ve geçişleri yönetir.
- **Ses Alt Sistemi:** Ses efektlerini ve arka plan müziğini yönetir.
- **Durum Yönetimi Sistemi:**
 - **Seri Hale Getirme Alt Sistemi:** Durumu JSON dosyalarına kaydeder ve yükler.
 - **Durum Erişim Alt Sistemi:** Durum değişkenleri için alıcılar ve ayarlayıcılar sağlar

3. Sınıf Tanımları

Aşağıda her sistem ve alt sistem için temel sınıflar, sorumlulukları ve özellikleri yer almaktadır.

- **Durum Yönetimi Sistemi:**

- **Sınıf:** GameState

- **Özellikler:**

- state: Oyunun durumunu saklayan sözlük (örneğin, sağlık, jetonlar, satın alınan yetenekler).
 - state_file: Durumun kalıcılığı için JSON dosyasının yolu.

- **Yöntemler:**

- save(): Durumu JSON'a seri hale getirir.
 - load(): Durumu JSON'dan ters seri hale getirir.

- **Varlık Yönetimi Sistemi:**

- **Sınıf:** AssetManager

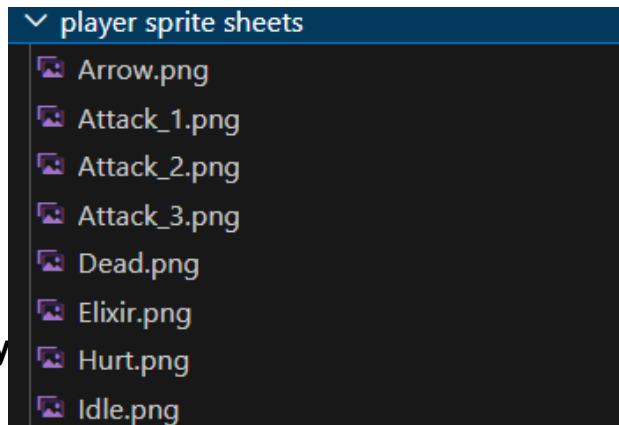
- **Özellikler:**

- assets_dir: Varlıkların dizin yolu.
 - image_cache: Yüklenen görüntülerin ön belleği.
 - sound_cache: Yüklenen seslerin ön belleği.

- **Yöntemler:**

- load_image(path, scale): Görüntüyü yükler ve ön belleğe alır, isteye bağlı ölçeklendirme yapar.
 - load_sound(path): Ses dosyasını yükler ve ön belleğe alır

- Oy



- **Sınıf:** GameManager
 - **Öznitelikler:**
 - game_state: GameState örneği durum yönetimi için.
 - current_dir: Oyun dosyalarının dizin yolu.
 - **Yöntemler:**
 - start_game(): Durumu kaydeder ve player.py'yi başlatır.
 - return_to_menu(): Durumu kaydeder ve menu.py'yi başlatır.
- **Ayarlar Sistemi:**
 - **Sınıf:** Settings
 - **Öznitelikler:**
 - game_state: Ayarlara erişim için GameState örneği.
 - **Yöntemler:**
 - apply_settings(screen): Görsel ayarları ekrana uygular.
 - set_volume(music_volume, sound_volume): Ses seviyelerini ayarlar.
- **Menü Sistemi:**
 - **Sınıf:** Menu
 - **Öznitelikler:**
 - asset_manager: AssetManager örneği.
 - game_manager: GameManager örneği.
 - current_tokens: Oyuncu jetonlarını takip eder.
 - shop_skills: ShopSkill nesnelerinin listesi.
 - current_screen: Aktif menü ekranını takip eder.
 - **Yöntemler:**
 - load_background(): Menü arka planını yükler.

- `start_game()`: GameManager ile oyun oynanışını başlatır.
 - `update()`: Kullanıcı girişine göre menü durumunu günceller.
- **Sınıf: PixelButton**
 - **Öznitelikler:**
 - `rect`: Düğmenin konumu ve boyutu.
 - `text`: Düğme etiketi.
 - `action`: Tıklama için geri çağrı fonksiyonu.
 - **Yöntemler:**
 - `update(mouse_pos, mouse_click)`: Düğme durumunu günceller (üzerinde gezinme, tıklama).
 - `draw(screen)`: Düğmeyi işler.
- **Oynanış Sistemi:**
 - **Sınıf: Samurai**
 - **Öznitelikler:**
 - `health, max_health`: Oyuncu sağlık istatistikleri.
 - `position`: Oyuncunun konumu (x, y).
 - `spritesheets`: Yürüme, zıplama, saldırı gibi animasyonlar için sprite'lar.
 - `attack_damage`: Saldırıların verdiği hasar.
 - **Yöntemler:**
 - `move(keys)`: Hareketi işler (yürüme, koşma, zıplama).
 - `attack()`: Normal veya şarjlı saldırı yapar.
 - `shoot_arrow()`: Ok atma gibi ekstra işlevler.
 - `Check_hit_enemies`: Düşmana saldırma ve gerekli verilen zarar.

```

469 > def jump(self): ...
482
483 > def attack(self, attack_type): ...
503
504 > def shoot(self, arrow_group): ...
518
519 > def check_hit_enemies(self, enemies): ...
539

```

- **Sınıf: NinjaMonk**

- **Öznitelikler:**

- health, max_health: Düşman sağlık istatistikleri.
- position: Düşmanın konumu (x, y).
- patrol_distance: Devriye davranışı için mesafe.
- attack_damage: Saldırının verdiği hasar.

- **Yöntemler:**

- patrol(): Belirli bir mesafede ileri geri hareket eder.
- attack(player): Oyuncuya saldırı yapar eğer mesafedeyse.
- update(player_pos): Oyuncu konumuna göre AI durumunu günceller.

```

1118 > def handle_collisions(self, dx, dy, collision_rects): ...
1143
1144 > def patrol(self, collision_rects): ...
1158
1159 > def update_attack_hitbox(self): ...
1164
1165 > def attack(self): ...
1173
1174 # Update the get_hit method:
1175 > def get_hit(self, damage): ...
1192
1193 # Update the detect_player method:
1194 > def detect_player(self, player): ...

```

- **Sınıf: Map**

- **Öznitelikler:**

- tmx_data: Yüklene Tiled harita verileri.
- collisions: Çarpışma nesnelerinin listesi.
- hazards: Tehlike bölgelerinin listesi.

- Transitions: Geçiş bölgelerinin listesi.
- **Yöntemler:**
 - load_map(path): Tiled haritasını yükler.
 - draw(screen, camera): Paralaks kaydırmayla haritayı işler.
 - find_spawn_point(spawn_name="spawn_point"): Karakterin bölümde oluşacağı konumu bulur.

4. Rutinler ve Arayüzler

Aşağıda sistem etkileşimleri için temel rutinler ve arayüzler yer almaktadır:

- **Durum Yönetimi Arayüzü:**
 - get_state(key: str) -> Any: Durum değerini alır.
 - set_state(key: str, value: Any): Durum değerini günceller.
 - **Uygulama:** GameState sınıfı tarafından sağlanır.
- **Varlık Yükleme Arayüzü:**
 - load_asset(asset_type: str, path: str, *args) -> Any: Varlık yükler (görüntü veya ses).
 - **Uygulama:**
 - AssetManager.load_image(path, scale) için görüntüler.
 - AssetManager.load_sound(path) için sesler.
- **Geçiş Arayüzü:**
 - transition_to(target: str): Başka bir sisteme geçiş yapar (menü veya oynanış).
 - **Uygulama:**
 - GameManager.start_game(): Oynanışa geçiş.
 - GameManager.return_to_menu(): Menüye dönüş.
- **Ayarlar Arayüzü:**
 - apply_game_settings(screen: pygame.Surface): Görsel ayarları uygular.

- `set_audio_settings(music_volume: float, sound_volume: float)`: Ses seviyelerini ayarlar.
- **Uygulama:**
 - `Settings.apply_settings(screen)`: Parlaklık ayarı.
 - `Settings.set_volume(music_volume, sound_volume)`: Ses seviyeleri.
- **Oynanış Güncelleme Rutini:**
 - `update_gameplay(dt: float, keys: dict, events: list)`: Her çerçeveyi günceller.
 - **Uygulama:**
 - `player.py`'deki ana döngü, `Samurai`, `NinjaMonk` ve `Map` nesnelerini günceller:

5. Dil, Çerçeve ve Kodlama Standartları

- **Dil:** Python 3.10
 - Basitliği, geniş kütüphane desteği ve potansiyel web dağıtımı için Pyodide uyumluluğu nedeniyle seçilmiştir.
- **Çerçeve:** Pygame 2.6.1
 - 2D işleme, ses yönetimi ve giriş yönetimi yetenekleri nedeniyle oyun geliştirme için kullanılmıştır.
- **Harici Kütüphaneler:**
 - `pytmx`: Tiled haritalarını yüklemek ve işlemek için.
 - `pygame.mixer`: Ses yönetimi için.
 - `JSON`: durum değişimlerini kaydetmek ve uygulamak için.
- **Kodlama Standartları:**
 - **Adlandırma Kuralları:**
 - Sınıflar: CamelCase (örneğin, `GameState`, `AssetManager`).

- Fonksiyonlar/Yöntemler: snake_case (örneğin, load_image, apply_settings).
- Değişkenler: snake_case
- **Kod Yapısı:**
 - Her modülün tek bir sorumluluğu olmalı.
 - Fonksiyon imzalarında tür ipuçları kullanılmalı.
 - Tüm sınıflar ve yöntemler için doküman dizeleri eklenmeli.
- **Hata Yönetimi:**
 - Varlık yükleme ve dosya işlemleri için try-except blokları kullanılmalı.
 - Yükleme başarısız olursa yedek varlıklar (örneğin varsayılan görüntü) sağlanmalı.
- **Dosya Organizasyonu:**
 - Tüm kaynak dosyalar src/ dizininde yer almalı.
 - Varlıklar assets/'te (örneğin, assets/menu_tasarim/, assets/sounds/).
 - Yapılandırma dosyaları (örneğin, game_state.json) kök dizinde.

6. Test Yaklaşımları

Oyunun güvenilirliği ve performansı için testler kritiktir. Aşağıdaki yaklaşımlar kullanılacaktır:

- **Birim Testleri:**
 - **Çerçeve:** unittest
 - **Odak:**
 - GameState seri hale getirme/ters seri hale getirme testleri.
 - AssetManager yedek davranış testi.
- **Entegrasyon Testleri:**
 - **Odak:**
 - menu.py ile player.py arasındaki geçişler için GameManager testi.
 - GameState'in sistemler arasında veri aktarımını doğru yapması.

- **Fonksiyonel Testler:**

- **Odak:**

- Oynanış mekanikleri testi.
 - Menü etkileşimlerini testi.

- **Performans Testleri:**

- **Odak:**

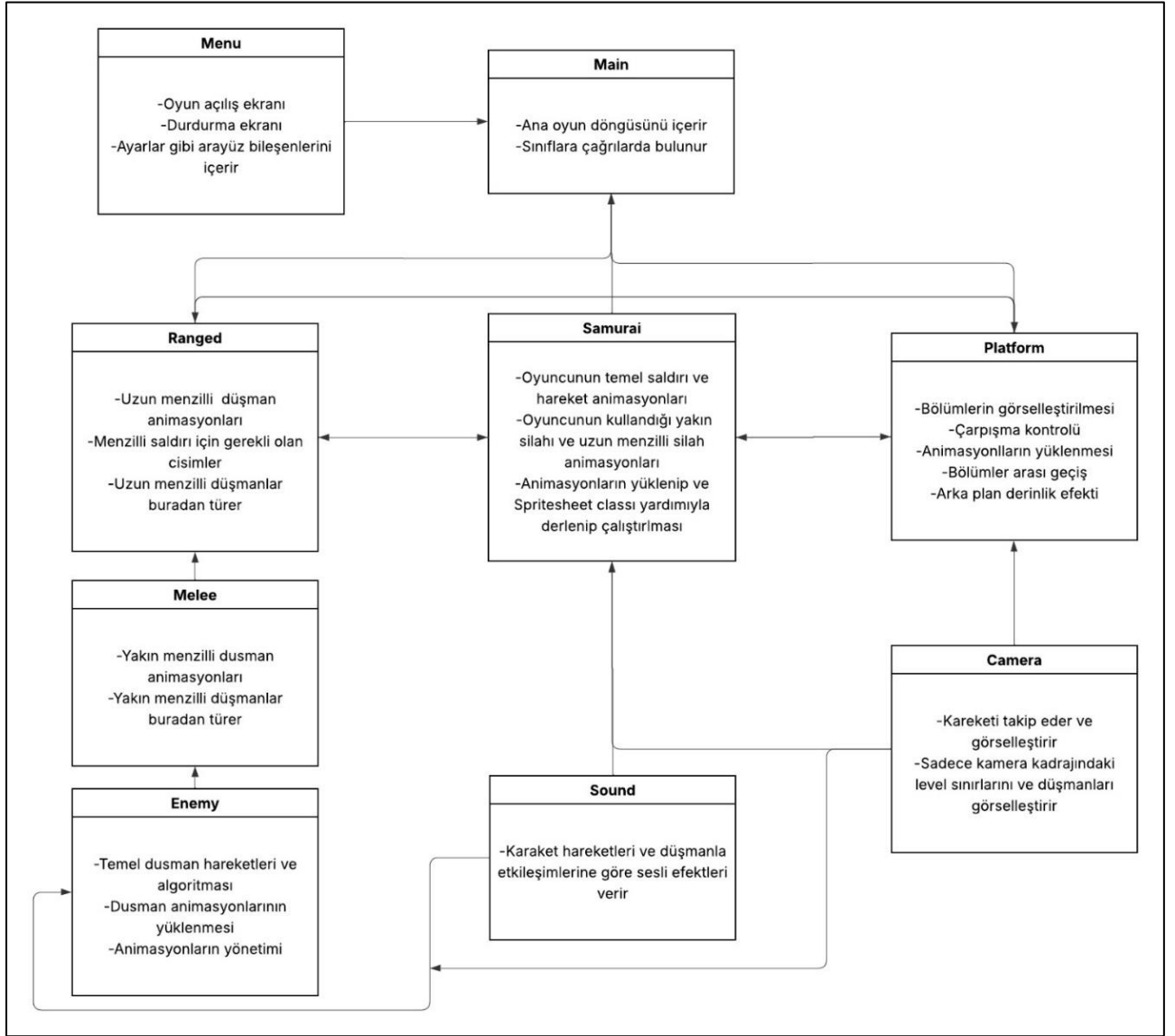
- Paralaks ön işleme ile ve olmadan kare hızı ölçümü.
 - AssetManager önbellekleme ile bellek kullanımını testi.

- **Araç:** FPS ölçümü için `pygame.time.Clock`:

- **Manuel Testler:**

- Kullanılabilirlik sorunlarını tanımlamak için oyunu oynayarak test et (örneğin menü gezintisi, oynanış tepkisi).
 - Kenar durumları test et (örneğin eksik varlıklar, geçersiz kullanıcı girişleri).

7.Mimari Diyagram



8. Sonuç

Bu mimari tasarım, "The Way" oyunu için modüler ve ölçeklenebilir bir çerçeve sağlar, menu.py, player.py, enemy.py, platform.py gibi birçok dosyayı birbirine entegre eden tutarlı bir sistem oluşturur. Tasarım, sorumlulukların ayrılmasını, durum yönetiminin tutarlılığını ve varlık önbellekleme ile ön işleme sayesinde optimize edilmiş performansı garanti eder. Belirtilen kodlama standartları ve test yaklaşımları, güvenilirlik ve sürdürülebilirliği sağlayacaktır. Yeni seviyeler, çok oyunculu özellikler gibi gelecekteki genişlemeler, mevcut modülleri genişleterek kolayca entegre edilebilir.